DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CSN3001: Data Structures Lab
Lab Assignment: Applications of Linked Lists

1. Objectives & General Guidelines

Linked lists provide fundamental building blocks for modeling ordered collections, cyclic queues, and sequential data streams. This introductory assignment focuses on two easy practical application problems designed to build confidence in manual pointer management:

- **Music Playlist Management:** Utilizing Circular Linked Lists to implement continuous track looping and seamless playback control.
- **Print Job Buffer Queue:** Utilizing Singly Linked Lists to construct a FIFO print spooler for adding and servicing print tasks.

2. Lab Problems

Problem 1: Music Player Playlist Looper

Design a music player playlist using a Circular Singly Linked List. The playlist supports adding new songs to the end, playing the current track, and skipping to the next track. Because it is circular, skipping past the last song seamlessly loops back to play the first song in the playlist.

Pointer Logic & Application Architecture:

```
Node Structure: Node { String songTitle; Node* next; }
playNext(): Move current pointer to current.next and output the songTitle.
addSong(title): Insert new node at the tail such that tail.next = head.
```

Sample Execution & System Interaction:

```
Input: addSong("Song A"); addSong("Song B"); addSong("Song C"); playNext() [x4]
Output: Playing: Song A -> Song B -> Song C -> Song A (Looped)
```

Constraints & Performance Bounds: *Number of songs N: 1 <= N <= 10^3. Operations playNext and addSong must run in O(1) time.*

Problem 2: Printer Queue Spooler System

Design a print spooler system using a Singly Linked List with head and tail pointers. The printer receives print jobs and processes them in First-In, First-Out (FIFO) order. Implement addJob(docName, pages) to enqueue a print request at the tail, processJob() to print and dequeue the job from the head, and getPendingCount() to count remaining jobs.

Pointer Logic & Application Architecture:

```
Node Structure: Node { String docName; int pages; Node* next; }
addJob(docName, pages): Append node at tail; tail = tail.next (O(1)).
processJob(): Remove head node; update head = head.next (O(1)).
```

Sample Execution & System Interaction:

```
Input: addJob("Lab_Report.pdf", 5); addJob("Notes.pdf", 2); processJob(); processJob()
Output: Printed: "Lab_Report.pdf" (5 pages) | Printed: "Notes.pdf" (2 pages) | Buffer Empty
```

Constraints & Performance Bounds: *Number of jobs: 1 <= N <= 10^4. Enqueue and Dequeue operations must strictly execute in O(1) time.*